

Vela: An Efficient Open-Source FPGA Hardware Accelerator for End to End Vision-Language-Action Inference

Cheng Nian, *Member, IEEE*, Jiaqi Zhang, Fasih Ud Din Farrukh, Jiaying Peng, Xiaorui Mo, Fei Chen, Chun Zhang, *Senior Member, IEEE*

Abstract—Low-bit arithmetic reduces the cost of vision-language-action (VLA) inference through compact data representation and dense matrix computation. Quantization groups also affect closed-loop task quality, while independently scaled partial sums and nonlinear operators require dedicated processing. This paper develops Vela, a W8A8 FPGA architecture with 64-element reduction groups. The design combines group-wise activation quantization and scale-aligned merging, DSP-pre-adder packing for two shared-activation products, and a heterogeneous nonlinear unit with shared elementwise and reduction paths. In 160 LIBERO episodes across eight tasks, the W8A8 G64 numerical implementation succeeds in 153 episodes, matching the floating-point total. Cycle-level evaluation gives 427.68 ms per policy invocation at 300 MHz, a 27.4% latency reduction relative to the same-resource FP16 reference. The 768-DSP matrix array provides a theoretical peak of 921.6 GOPS. Post-route module results quantify the resource and power costs of quantization, merging, packed arithmetic, and nonlinear execution. The architecture simulator and reproduction package are available at <https://github.com/nc-thu/vela>.

Keywords: vision-language-action, FPGA, group quantization, DSP packing, nonlinear acceleration.

I. INTRODUCTION

Vision-language-action policies translate observations and instructions into robot actions. RT-2 transfers web-scale vision-language knowledge to robotic control [1], OpenVLA provides an open generalist manipulation model [2], and π_0 couples pretrained vision-language representations to flow-based action generation [3]. Their execution combines visual processing, interaction between modalities, and action prediction. TurboVLA reduces the dependence on a large language model by encoding vision and language separately and exchanging information through lightweight interaction modules [4]. The resulting workload still contains matrix operations, numerical conversions, nonlinear functions, and reductions. An FPGA implementation must support this mixture rather than optimize only an isolated dense layer.

Low-bit computation offers two immediate opportunities. Fewer weight bits reduce the amount of parameter data to

move, and multiple narrow products can share a wider DSP multiplier. However, the number of bits also determines how accurately the policy represents its inputs and intermediate values. In a closed-loop robot task, numerical changes can alter later observations and decisions. The relevant design objective is therefore to reduce execution cost while preserving useful task performance, not simply to maximize products per DSP.

Three observations guide this work. First, quantization granularity strongly affects quality. In the evaluated selection set, changing W4A8 from a 64-element reduction group to a 32-element group increases success from 111/160 to 144/160. Smaller groups introduce more independently scaled partial sums, which the hardware must process. The same parameter that improves numerical resolution also changes the volume and frequency of merge work.

Second, packed multiplication requires support from the surrounding datapath. Pack2 produces two W8A8 products per DSP cycle, but the array must also recover signed fields, accumulate each group, and transfer its results for scale-aligned merging. In the W8A8 G64 model, matrix products account for 31.9% of critical-path cycles, while data movement and layout account for 33.2%. These costs motivate attention to operand delivery and the interfaces between packed arithmetic, group-result storage, and merging.

Third, nonlinear work remains substantial after matrix acceleration. In the W8A8 execution schedule, the nonlinear category accounts for 20.14% of critical-path cycles. Elementwise activations and reduction-dependent functions also have different execution requirements. Normalization introduces serial dependencies, while exponentials and reciprocal operations consume separate arithmetic resources.

Existing approaches address parts of this problem. SmoothQuant redistributes quantization difficulty through an equivalent channel-wise transformation [5]. AWQ uses activation information to guide low-bit weight quantization [6]. DSP packing increases the arithmetic density of FPGA multipliers, with explicit attention to errors and corrections [7]. These mechanisms motivate useful arithmetic choices, but a grouped VLA datapath must additionally coordinate scale handling, partial-sum production, and heterogeneous operators.

The proposed architecture organizes these requirements into three hardware components. H1 provides the grouped quantization framework: it quantizes activations in reduction groups, buffers group results, and merges them using aligned scale coefficients. Its system evaluation compares the complete

This work was supported in part by the Development and Reform Commission of Shenzhen Municipality under Grant XMHT20220115007. (Corresponding author: Chun Zhang.)

Cheng Nian and Fei Chen are with the Greater Bay Area National Center of Technology Innovation, Research Institute of Tsinghua University in Shenzhen, Shenzhen 518057, China (e-mail: nc23@mails.tsinghua.edu.cn).

Jiaqi Zhang, Fasih Ud Din Farrukh, Jiaying Peng, Xiaorui Mo, and Chun Zhang are with the School of Integrated Circuits, Tsinghua University, Beijing 100084, China (e-mail: zhangchun@tsinghua.edu.cn).

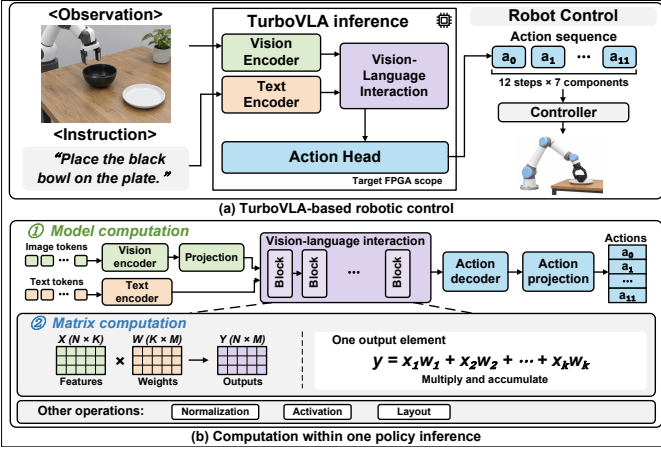


Fig. 1. TurboVLA workload. A policy invocation maps observations and instructions to a chunk of 12 seven-dimensional actions.

quantized architecture with a matched-resource FP16 array model. H2 maps two shared-activation INT8 products to each DSP pre-adder and multiplier using Pack2. H3 is a heterogeneous nonlinear unit (HNU) that combines piecewise-linear elementwise evaluation with reduction and special-function paths. Static-weight products and dynamic attention both use eight-bit operands.

The contributions are a workload-driven analysis of quantization granularity, an explicit W8A8 arithmetic mapping from grouped values to packed products and scale-aligned outputs, and an FPGA architecture study supported by closed-loop numerical evaluation, cycle modeling, and module implementation. The quality comparison motivates W8A8 G64 as the hardware configuration. The complete design reduces modeled invocation latency by 27.4% relative to the matched-resource FP16 array reference. Mechanism comparisons separately examine the snapshot/merge organization, Pack2 density, and shared HNU datapath.

II. WORKLOAD AND DESIGN CHALLENGES

A. VLA model organization

VLA models differ in how semantic representations drive robot actions. CogACT conditions a specialized diffusion action transformer on vision-language features [8], while DiTA scales a diffusion-transformer policy for generalist control [9]. SpatialVLA introduces 3D position information and adaptive spatial action grids [10]. SmolVLA emphasizes a compact model and decouples action prediction from execution through an asynchronous inference stack [11]. These designs make model organization and action execution part of the deployment problem. Vela studies the operator mixture of TurboVLA and selects arithmetic granularity using closed-loop task quality.

B. Execution scope and precision notation

Figure 1 places the arithmetic in the policy workflow. The architecture trace contains 2,277 operators and 52,673,989,632 multiply-accumulate operations (MACs) per W8A8 G64 invocation. An invocation produces a chunk of 12 seven-dimensional

actions. The execution policy consumes these actions and determines when to request the next invocation.

W8A8 denotes eight-bit static weights and eight-bit activations. The quality experiments also evaluate W4A8, which uses four-bit static weights and eight-bit activations. In attention products with two dynamic operands, both inputs remain eight-bit. The notation G64 or G32 specifies how many elements along the matrix reduction dimension K share a scale. The hardware configuration uses W8A8 G64.

For an operation $Y = AB$, partitioning K into G -element groups produces independently quantized subproducts. Activations require runtime statistics. Constant weights can be prepared offline, whereas a dynamic second operand needs runtime preparation. Both paths supply INT8 operands to the Pack2 datapath, while their scale metadata is prepared at different times.

C. C1: Quality-sensitive grouping creates merge work

The nine-configuration quality study compares the same eight tasks and 20 initial states per task. Its Direct baseline uses a tensor-wide activation scale and output-channel static-weight scales, without K grouping. W8 Direct succeeds in 57/160 episodes, while W8 G64 succeeds in 153/160. W4 Direct succeeds in one episode. W4 G32 succeeds in 144. These differences establish that the representation granularity is consequential for this policy and selection set.

The useful group size also depends on precision. W4 G64 and G128 succeed in 111 and 105 episodes, respectively, below W4 G32. W8 G32 reaches 151 successes, compared with 153 for G64 and 152 for G128. These results motivate G64 for the W8 hardware configuration and demonstrate greater grouping sensitivity at four-bit weight precision.

Grouping also changes hardware demand. Each output element receives approximately K/G partial sums, and each partial sum carries its own product of operand scales. Halving G approximately doubles the number of group boundaries for a fixed, divisible K . The partial sums cannot be added as unscaled integers when those scales differ. Hardware must align them, accumulate them, and preserve sufficient numerical range.

An earlier schedule with a narrower merge organization illustrates the cost of group-result handling: changing W8 from G64 to G32 increases its critical-path merge contribution from 656,798 to 64,571,430 cycles. This observation motivates separating matrix production from group-result consumption and increasing the merge service rate.

D. C2: Packed products require system support

For shared activations, Pack2 generates two W8A8 products per DSP cycle. Sustaining this density requires input delivery, signed-field recovery, accumulator state, and downstream processing of completed groups.

The 48 physical columns expose 96 logical output columns under Pack2. At each group boundary, completed partial sums must be retained until their scale coefficients and merge lanes are available. If the merge or storage path cannot accept the results, the matrix array waits. The hardware must therefore

coordinate packed multiplication with result storage and merge service.

In the W8A8 G64 invocation, matrix products occupy 40,873,040 critical-path cycles out of 128,304,013. Data movement and layout occupy another 42,626,787 cycles. These costs motivate coordinating packed arithmetic with operand delivery, snapshot storage, and group merging.

E. C3: Nonlinear and reduction work remains visible

The W8A8 G64 configuration assigns 25,833,842 critical-path cycles to the nonlinear category, corresponding to 86.11 ms at 300 MHz. The workload includes 20,366,568 Softmax input values, 23,722,512 LayerNorm input values, and 21,066,240 GELU input values per invocation.

GELU-like elementwise functions can be evaluated through independent coefficient lookups and multiply-adds. Softmax needs a row reduction, exponentials, another reduction, and normalization. LayerNorm similarly depends on statistics shared across a row. These dependencies motivate H3's division into elementwise, reduction, and special-function resources.

Figure 2 connects the workload observations to the hardware responses. The three challenges concern successive parts of one computation: representing inputs, producing integer products, and processing results and nonlinear operators. The design coordinates the service rates and buffering of these stages.

III. NUMERICAL METHOD AND ARITHMETIC MAPPING

A. Group-wise INT8 representation

Let $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$. For output indices m, n and reduction group g , define $\mathcal{K}_g$ as the set of reduction indices in that group. For a vector x and bit width b , the numerical reference uses

$$L_b = 2^{b-1} - 1, \quad (1)$$

$$s_x = \max(\max_i |x_i|, \epsilon) / L_b, \quad (2)$$

$$q_i = \text{clip}(\text{RNE}(x_i / s_x), -L_b, L_b), \quad (3)$$

where $\epsilon = 10^{-12}$ in the software reference and RNE denotes round-to-nearest with ties to even. The W8A8 datapath uses $b = 8$ and $L_8 = 127$ for weights and activations. Zero groups encode to zero.

The group product and its scale are

$$p_{mng} = \sum_{k \in \mathcal{K}_g} q_{mkg}^A q_{kng}^B, \quad (4)$$

$$s_{mng} = s_{mg}^A s_{ng}^B, \quad (5)$$

$$Y_{mn} \approx \sum_g s_{mng} p_{mng}. \quad (6)$$

Equation (6) uses row-, column-, and group-dependent scales. Padding a final incomplete group with zeros preserves the matrix dimensions after the padded positions are discarded.

B. Common-exponent scale alignment

Figure 3 separates integer products from scale metadata. A common exponent E permits group scales to be represented as integer coefficients. The software reference obtains E from the `frexp` exponent of the maximum scale product in the operation and forms

$$c_{mng} = \text{RNE}(s_{mng} 2^{24-E}), \quad (7)$$

$$Z_{mn} = \sum_g p_{mng} c_{mng}, \quad (8)$$

$$\hat{Y}_{mn} = Z_{mn} 2^{E-24}. \quad (9)$$

The exponent is retained with the result. Coefficients must be available before their corresponding partial sums are consumed. This preparation is distinct from the dot product itself, allowing the architecture to schedule scale handling separately from matrix arithmetic.

This representation introduces a coefficient-rounding error in addition to operand quantization. Before considering accumulator overflow, the coefficient contribution is bounded by

$$\left| \sum_g p_g (s_g - c_g 2^{E-24}) \right| \leq 2^{E-25} \sum_g |p_g|. \quad (10)$$

This coefficient-error bound follows from nearest rounding and determines the required exponent, coefficient range, and integer accumulation width.

The quality evaluation accumulates (8) in INT64. The implemented merge primitive accepts a signed 17-bit dot product and a signed 24-bit coefficient, and accumulates in 48 bits. Mapping the reference computation to this interface requires range adaptation for group sums and coefficients; a full W8 G64 dot product can exceed the signed 17-bit input range.

C. Output representation

The FP16 path restores the common scale and rounds once to binary16:

$$y_{mn} = \text{FP16}(Z_{mn} 2^{E-24}). \quad (11)$$

The dequantization RTL normalizes the integer magnitude, forms the exponent, and uses guard, round, and sticky information for nearest-even rounding. It handles normal and subnormal outputs, signed zero, and overflow to infinity. FP16 results can then enter vector operations and the HNU without inserting an INT8 re-encoding stage.

The optional integer stream instead buffers Z and performs two passes. The first obtains the maximum magnitude P_h within output group h . The second computes

$$q_h = \text{RNE}(127 Z_h / P_h), \quad s_h^{\text{out}} = \frac{P_h}{127} 2^{E-24}. \quad (12)$$

A zero group produces zero output. Ordinary matrix outputs use 32-value groups along each output row. The action projection groups along time for the seven action coordinates. The second pass needs both the buffered values and their group maximum; it cannot be implemented from the maximum alone.

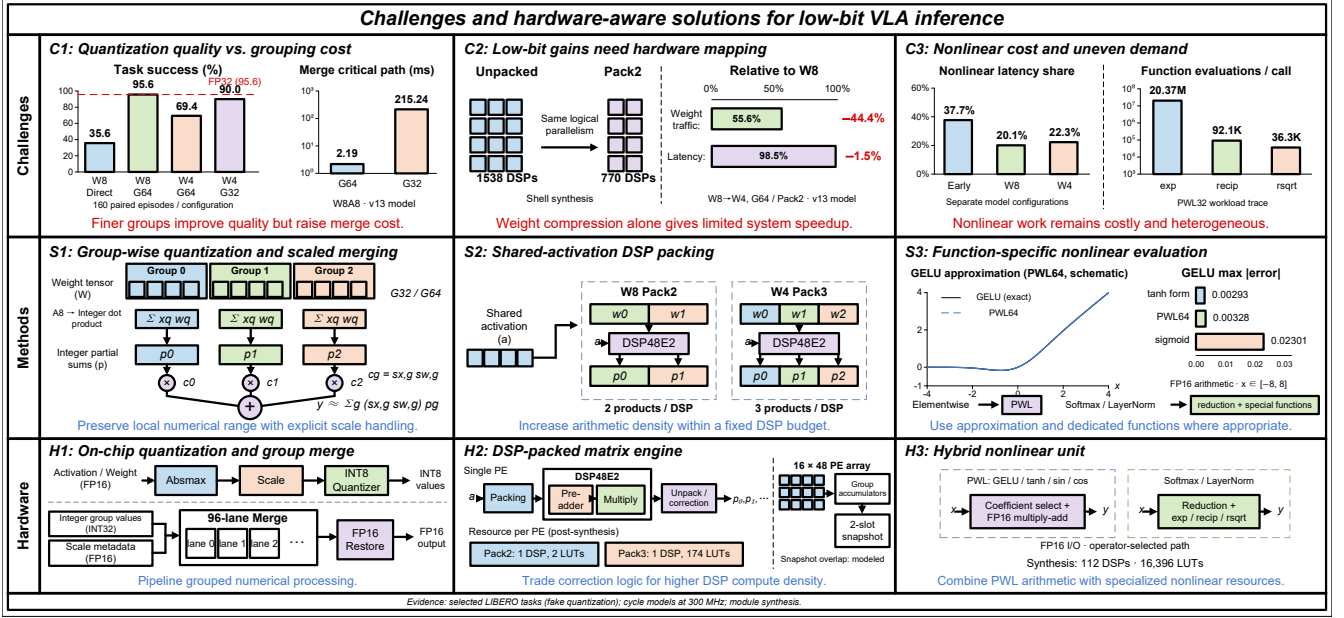


Fig. 2. Workload challenges and hardware responses: C1/H1 quantization and merging, C2/H2 DSP-packed arithmetic, and C3/H3 nonlinear execution.

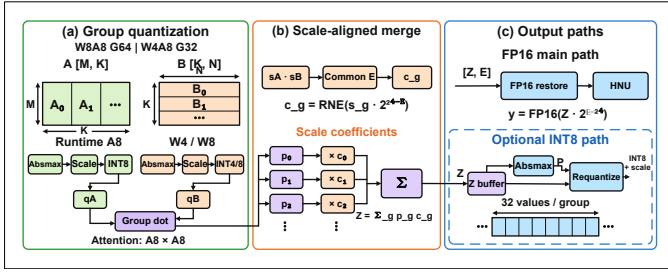


Fig. 3. H1 group-wise quantization algorithm. Inputs are grouped along K , group products are merged with aligned coefficients, and the output enters either FP16 restoration or integer re-encoding.

D. Pack2: two signed W8A8 products

Let signed eight-bit weights w_0, w_1 share a signed activation a . Offset encoding changes each weight to a nonnegative byte, implemented by flipping its sign bit. The packed word is

$$Q = (w_1 + 128)2^{16} + (w_0 + 128). \quad (13)$$

The DSP pre-adder subtracts the constant $128(2^{16} + 1) = 8,388,736$ before multiplication, yielding

$$R = w_12^{16} + w_0, \quad P = aR. \quad (14)$$

This placement removes the activation-dependent offset correction from the post-multiply path. Signed products are recovered as

$$p_0 = \text{signed}(P[15 : 0]), \quad (15)$$

$$p_1 = \text{signed}(P[31 : 16]) + P[15]. \quad (16)$$

The additional bit in p_1 corrects the borrow caused by a negative lower field. Omitting it would couple the numerical interpretation of the high field to the sign of the low product.

Pack2 uses one DSP per packed multiplier. Signed-product recovery feeds the downstream group accumulators.

E. Nonlinear numerical paths

For a piecewise-linear (PWL) function, interval selection returns a slope α_j and intercept β_j , and the elementwise path computes $f(x) \approx \alpha_j x + \beta_j$. The HNU uses a 64-segment organization for supported functions. Sharing the multiply-add structure across functions reduces duplicated arithmetic, while coefficient tables select the function-specific approximation.

Softmax and LayerNorm additionally require reductions. A stable Softmax implementation forms a row maximum, evaluates exponentials of shifted inputs, sums them, and multiplies by the reciprocal sum. LayerNorm computes normalization statistics and uses reciprocal square root. The architecture retains FP32 reduction state and FP16 interfaces, with distinct resources for these special functions.

IV. FPGA ARCHITECTURE

A. Dataflow overview

The architecture follows the numerical dependencies in Section III. DMA transfers inputs and parameters between external memory and on-chip banks. Runtime quantization produces grouped activation records. The matrix array consumes quantized operands, and group results enter snapshot storage before scale-aligned merging. The completed result is restored to FP16 for vector and nonlinear processing, or encoded for a compatible integer consumer.

Figure 4 provides the module-level view. Compiler-generated execution plans supply operator order, addresses, grouping, and output-format choices. The controller coordinates these decisions rather than introducing another numerical approximation. Read ports and a DMA path support overlap between data movement and execution where dependencies permit it. Resource availability and buffer ownership remain explicit constraints on that overlap.

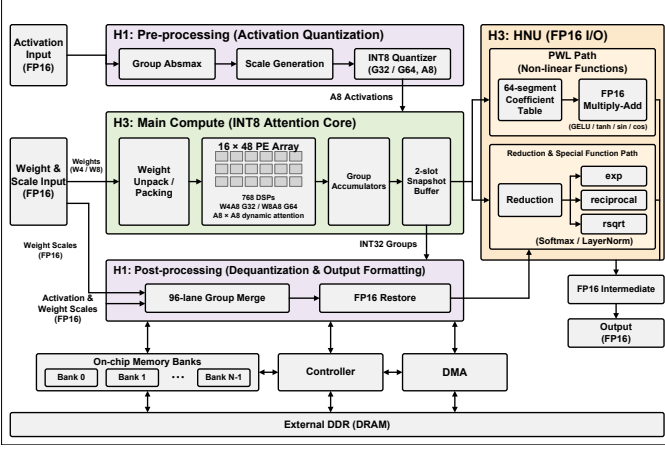


Fig. 4. Vela architecture. H1 performs input quantization and group merging, H2 executes packed matrix arithmetic, and H3 handles nonlinear and reduction operations. Scale restoration supplies FP16 consumers with merged results.

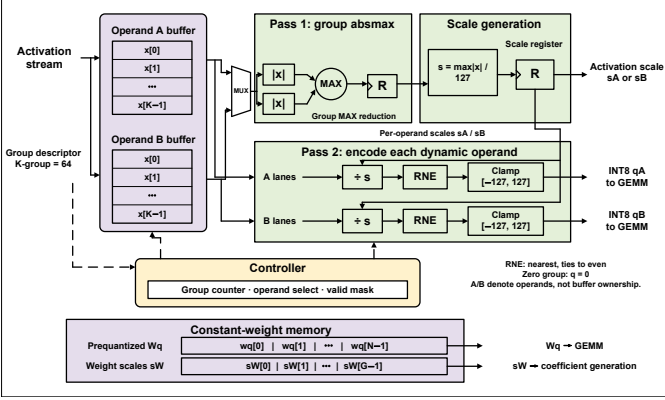


Fig. 5. Input quantization datapath. The parameterized implementation supports G32 and G64 at 32 FP16 input values per beat.

B. H1: Runtime quantization

The activation quantizer processes 32 FP16 values per input beat. G32 forms one group from one beat; G64 forms a group from two consecutive valid beats. Group absmax determines shared parameters, while elementwise pipelines produce signed INT8 values. The G64 pair must arrive consecutively within a group, although idle cycles are allowed between groups.

Figure 5 separates group statistics from element processing. The implementation uses reciprocal-related parameters and integer correction to reproduce the reference rounding behavior, including half-integer cases. Parameters and element data travel through aligned pipelines so that every quantized value uses its own group’s maximum.

The output record contains G quantized bytes and an eight-byte descriptor. The record sizes are therefore 40 bytes for G32 and 72 bytes for G64. Relative to two-byte FP16 input values, these correspond to 37.5% and 43.75% smaller activation records, respectively, before considering other transfers or buffering. The larger metadata fraction at G32 illustrates a second cost of fine grouping beyond the number of group results.

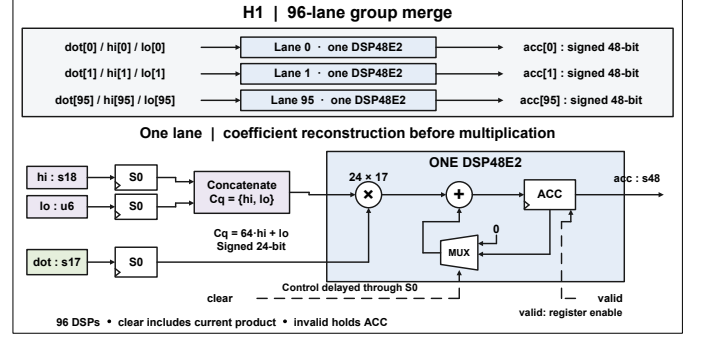


Fig. 6. Updated merge96 datapath. Each lane receives its own dot product and coefficient. Coefficient concatenation precedes multiplication; the lane then performs one DSP multiply-accumulate into a signed 48-bit result.

C. H1: Snapshots and 96-lane merging

The architecture model places two snapshot slots between group-result production and merge consumption. A completed group can occupy one slot while the merger consumes the other. The intended ownership rule is that the array writes only a free slot and the merger reads only a completed slot. A slot becomes reusable after its results have been consumed. If both slots are occupied, production must wait.

Let T_p denote the average production interval and T_m the time needed to consume a group snapshot. Sustained overlap requires $T_m \leq T_p$; otherwise the two slots eventually fill and backpressure stalls production. This service-rate condition motivates the 96-lane merge width. The cycle-level scheduler tracks slot ownership and backpressure explicitly.

Figure 6 shows the implemented merge primitive. Split coefficient metadata contains signed high bits h and unsigned low bits l . Their reconstruction is

$$C_q = 64h + l = \{h, l\}. \quad (17)$$

Because the fields do not overlap, concatenation implements the reconstruction without an adder. This moves the split from the multiplication side to the coefficient side. Each lane then fits a 24-by-17-bit product and 48-bit accumulation into one DSP.

The lane’s registered valid signal enables an update. Its clear signal selects zero instead of the previous accumulator, while still adding the current product. Invalid cycles hold the accumulator. A 96-lane instance broadcasts valid and clear but retains independent data and coefficients for each lane. The earlier split-multiplication implementation used 288 DSPs; coefficient-side reconstruction uses 96, a 66.7% reduction for this module.

D. H2: Packed arithmetic and array organization

Figure 7 shows the Pack2 mapping to the DSP48E2 pre-adder and multiplier. Pack2 accepts two offset W8 fields and the sign-extended A8 operand. The DSP output is registered, with control information delayed alongside the data before signed-product recovery and group accumulation.

The matrix architecture uses a 16×48 physical PE organization with 768 matrix DSPs. Pack2 provides 96 logical columns.

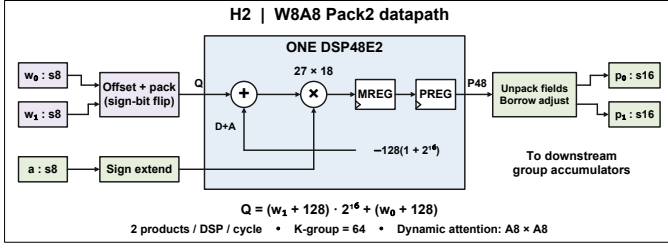


Fig. 7. H2 Pack2 circuit datapath. Two separate W8 operands share an A8 input. The DSP pre-adder removes the packing offset before multiplication; MREG and PREG register the packed result. Field extraction recovers two signed products for downstream group accumulation.

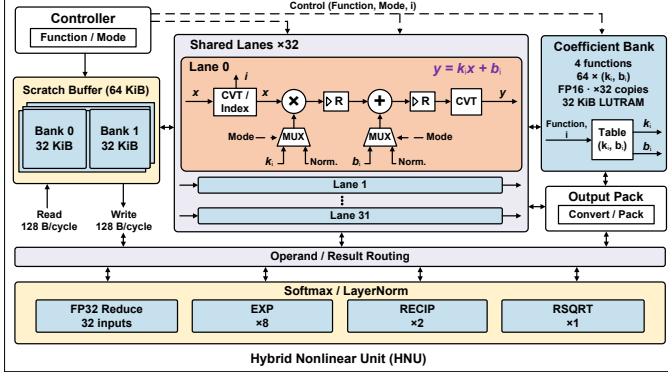


Fig. 8. HNU datapath with 32 shared PWL lanes, FP32 reduction state, eight exponential units, two reciprocal units, and one reciprocal-square-root unit.

Output tiling and scheduling determine how group results enter the snapshot slots and 96 merge lanes.

Dynamic attention uses the same Pack2 arithmetic for A8-by-A8 products. Its operands are quantized at runtime, while static W8 parameters can be prepared offline. Both feed the shared group-result and scale-aligned merge path.

E. H3: Heterogeneous nonlinear execution

The HNU divides its work between a 32-lane shared elementwise path, FP32 reductions, and special-function units, as shown in Fig. 8. A 64-segment coefficient table supplies the slope and intercept for PWL evaluation. The elementwise lanes reuse multiply-add arithmetic across supported nonlinear functions.

The special-function mix reflects differing service demands. Exponentials are consumed across Softmax elements, whereas reciprocal and reciprocal-square-root results are shared by a normalized row. The architecture therefore allocates more lanes to exponential evaluation than to row-level reciprocal operations.

Reduction state accumulates values across a row before dependent operations are released. The row maximum and sum in Softmax, for example, determine when exponentiation and normalization can proceed. The controller must preserve these dependencies even when independent elementwise work is available. This is the reason H3 includes both arithmetic paths and scheduling state, rather than a single lookup-and-multiply block.

F. Memory and execution control

Grouped data records couple values with descriptors, allowing consumers to identify group boundaries and interpret scales. The controller associates each scheduled operation with its input representation and selected output path. Buffers cannot be reused simply because a matrix tile has finished computing; snapshots, coefficient consumers, and vector operations may still reference their contents.

Read arbitration, local movement, and DMA resources track these data dependencies. The execution schedule overlaps data transfers with arithmetic and attributes invocation cycles to the critical path.

V. EVALUATION

A. Experimental setup

The quality study uses a fixed TurboVLA LIBERO checkpoint with EMA parameters and the `libero_all4_no_noops` preprocessing statistics. It evaluates task indices 0 and 1 from each of the LIBERO Spatial, Object, Goal, and Long suites [12], using initial states 3–22 and seed 7 + state_id. Each configuration is evaluated on 160 episodes across these eight tasks. Three successful W8 episodes illustrate the action sequences.

Closed-loop quality is measured with the software numerical implementation: Q24/INT64 merging, nearest-even output encoding, and floating-point auxiliary operations. The Direct configuration changes scale granularity while keeping the evaluation protocol fixed. Hardware evaluation covers the FP16 restoration, HNU, and merge primitives described in Sections III and IV.

Latency is evaluated using a cycle-level event model of the 2,277-operator W8A8 G64 invocation at 300 MHz. Module resources, timing, and power are obtained with Vivado 2021.2 on an `xczu7ev-ffvc1156-2-e` device. Each module undergoes out-of-context synthesis, optimization, placement, physical optimization, and routing under a 3.333 ns clock constraint. This flow characterizes individual modules before top-level clock-tree integration.

Power uses post-route vectorless activity estimation. The W8 multiplier wrapper covers 768 product cores and their input registers; downstream group accumulation lies outside this reported hierarchy. Aggregate estimates sum the listed modules with one G64 quantizer, one G32 output encoder, DMA included in the memory subsystem, and one device-static contribution. The ablation DSP budget comprises the matrix, merge, HNU, and controller resources; the module subtotal additionally includes 97 quantizer DSPs and one encoder DSP.

B. Closed-loop quality

Table I and Fig. 9 show the precision-group tradeoff. W8 G64 matches the FP32 total of 153 successes. W4 G32 reaches 144 successes, a reduction of 5.625 percentage points.

Fine grouping is particularly important for W4. Moving from G64 to G32 improves success by 20.625 percentage points, or 29.73% relative to the G64 rate. W8 exhibits much smaller differences across the grouped configurations. Thus, using the

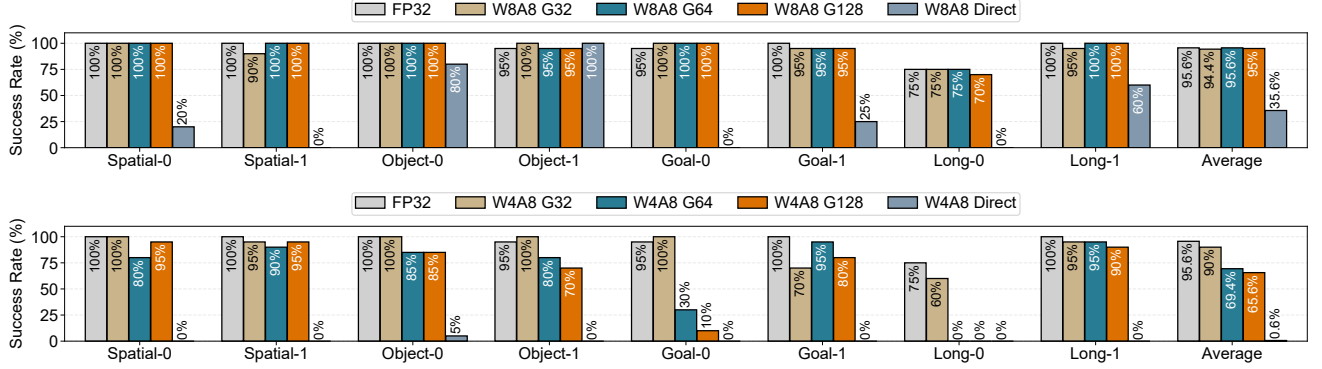


Fig. 9. Closed-loop success rates for different weight precisions and group sizes.

TABLE I
CLOSED-LOOP QUALITY ACROSS EIGHT LIBERO TASKS AND 160 EPISODES.

Configuration	Successes	Rate (%)	Δ FP32 (pp)
FP32	153	95.625	0
W8 Direct	57	35.625	-60.000
W8 G32	151	94.375	-1.250
W8 G64	153	95.625	0
W8 G128	152	95.000	-0.625
W4 Direct	1	0.625	-95.000
W4 G32	144	90.000	-5.625
W4 G64	111	69.375	-26.250
W4 G128	105	65.625	-30.000

TABLE II
SUCCESSSES PER SELECTED TASK; 20 EPISODES EACH.

Task	FP32	W8 G64	W4 G32
Spatial 0	20	20	20
Spatial 1	20	20	19
Object 0	20	20	20
Object 1	19	19	20
Goal 0	19	20	20
Goal 1	20	19	14
Long 0	15	15	12
Long 1	20	20	19

same group size for both weight formats would hide a material part of the design space.

Task-level results further qualify the aggregate. Table II shows that W4 G32 succeeds in all 40 Object episodes, but loses more episodes in Goal and Long tasks. An Object-only comparison would therefore overstate its quality across the selected workloads. The W8 operating point also exchanges successes between the two Goal tasks despite matching the floating-point aggregate.

Figure 10 illustrates three successful W8 episodes, showing the policy-controlled sequence from initial observation to task completion in simulation.

C. Inference latency

Table III reports the complete W8A8 G64 invocation schedule: 128,304,013 cycles, or 427.680 ms at the model

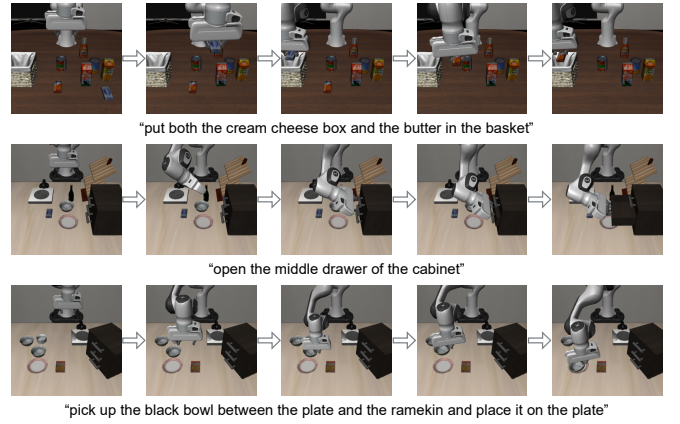


Fig. 10. Representative W8A8 G64 closed-loop rollouts in LIBERO simulation.

TABLE III
W8A8 G64 INFERENCE LATENCY AT 300 MHz.

Metric	W8 G64
Total cycles (million)	128.304
Latency (ms)	427.680
Nonlinear critical cycles (million)	25.834
Nonlinear critical share (%)	20.14

clock. The schedule includes data movement, group processing, and nonlinear computation in addition to packed multiplication.

Nonlinear operations account for 20.14% of the critical-path cycles. Their service time remains an important optimization target alongside the matrix path, motivating the dedicated function resources in H3.

D. Framework and mechanism comparisons

Figure 11 evaluates the complete design against four references in the cycle-level model. The comparisons use the same workload trace and memory configuration at 300 MHz. The FP16 reference changes the numerical representation and bypasses quantization; the three mechanism references retain W8A8 G64. Each reported reduction is $(T_{\text{ref}} - T_{\text{full}})/T_{\text{ref}}$.

H1: grouped quantization framework. The matched-resource FP16 array reference requires 589.3 ms, compared

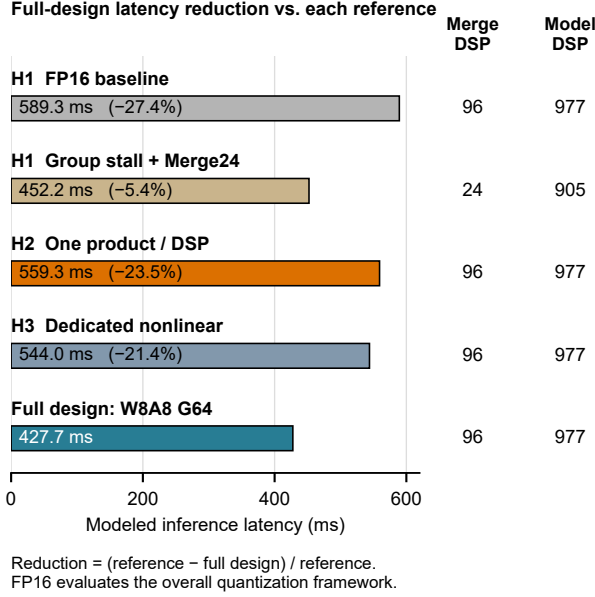


Fig. 11. Framework and mechanism comparisons at 300 MHz. Percentages denote full-design latency reductions relative to each row. H1 includes FP16 execution and group-stall/Merge24 references; H2 uses one product per DSP; H3 uses dedicated nonlinear units. DSP columns give merge resources and the ablation resource budget.

with 427.7 ms for the complete W8A8 G64 design, a 27.4% reduction. This comparison measures the overall benefit of quantized execution, including the packed arithmetic enabled by INT8 operands.

The second H1 reference removes double-buffered group snapshots, stalls the array at group boundaries, and reduces merge width from 96 to 24 lanes. Its latency is 452.2 ms, so the complete organization reduces latency by 5.4%. The rollback uses 24 merge DSPs and reduces the ablation resource budget from 977 to 905 DSPs (7.4%), exposing the latency-resource tradeoff. The overlap-only check retains Merge96 and takes 450.7 ms, corresponding to a 5.1% full-design reduction; retaining snapshots with Merge24 takes 429.2 ms. These checks identify overlap as the main latency contributor at G64.

H2: packed multiplication. With one INT8 product per DSP cycle, invocation latency is 559.3 ms. Pack2 reduces this to 427.7 ms, a 23.5% reduction under the same recorded DSP budget. This system improvement is smaller than the twofold product density because group processing, nonlinear execution, and data movement remain in the schedule.

H3: shared nonlinear datapath. The dedicated-function reference partitions the nonlinear processing capacity among functions instead of sharing the HNU datapath. Under its matched resource budget, it requires 544.0 ms; the complete HNU reduces invocation latency by 21.4%. Sharing allocates arithmetic capacity across successive functions according to their execution demand.

E. Merge implementation

The expression $(d \ll 6)h + dl$ creates multiple multiplication structures in the earlier merge implementation. Reconstructing

$C_q = \{h, l\}$ before multiplication maps each lane to one DSP MAC, reducing Merge96 from 288 to 96 DSPs, a 66.7% saving.

F. Implementation resources and power

Table IV lists selected module results after routing. Merge96 uses 96 DSPs and has 1.459 ns worst setup slack. The HNU uses 112 DSPs, 16,027 LUTs, and eight BRAM36-equivalent blocks, with 0.491 W estimated dynamic power. Quantization uses 97 DSPs in either group configuration. These costs show why non-matrix components belong in an FPGA resource budget.

The W8 wrapper closes timing at +0.548 ns after inserting 32 replicated input-register groups, each of which drives 24 PEs. This change limits the fanout of operands broadcast to 768 DSPs without modifying the Pack2 multiplier core. LUT use rises from 15 to 384 and FF use from 37,081 to 38,784, while the vectorless dynamic-power estimate changes from 1.939 to 1.926 W.

The implemented four-port read arbiter provides request and response backpressure, write-priority arbitration, and 16 physical banks. Each bank contains 32 entries of 128 bytes, for 64 KiB in total. Verification runs 20,101 cycles for each of four random seeds and checks request preservation, unique responses, and forward progress under backpressure. Its +0.010 ns OOC margin is the tightest entry in the table.

The W8 module subtotal is 78,147 LUTs and 1,075 DSPs. The listed dynamic-power estimates sum to 5.428 W; adding one 0.60 W device-static contribution gives approximately 6.03 W.

The W8A8 G64 results in Fig. 12 connect implementation costs to execution time. Matrix products account for 31.9% of critical-path cycles and group processing accounts for 6.7%. HNU work, including nonlinear, reduction, and elementwise operations, accounts for 28.2%; data movement and layout account for 33.2%. The complete invocation takes 427.68 ms.

G. Operating-point interpretation

W8A8 G64 is the selected hardware operating point. Its numerical reference matches the floating-point selection-set total, while G64 produces fewer reduction groups than G32. The resource and schedule results show that scale handling, nonlinear work, and data movement must be considered together with packed multiplication when implementing this configuration.

VI. RELATED WORK AND DISCUSSION

A. Comparison with FPGA accelerators

Table V compares FPGA accelerators for vision transformers, language models, and robot action planning. The reported configurations span INT8 vision processing, mixed-precision language inference, and INT8 action generation. Vela targets the complete TurboVLA policy invocation, including visual processing, cross-modal interaction, and action prediction.

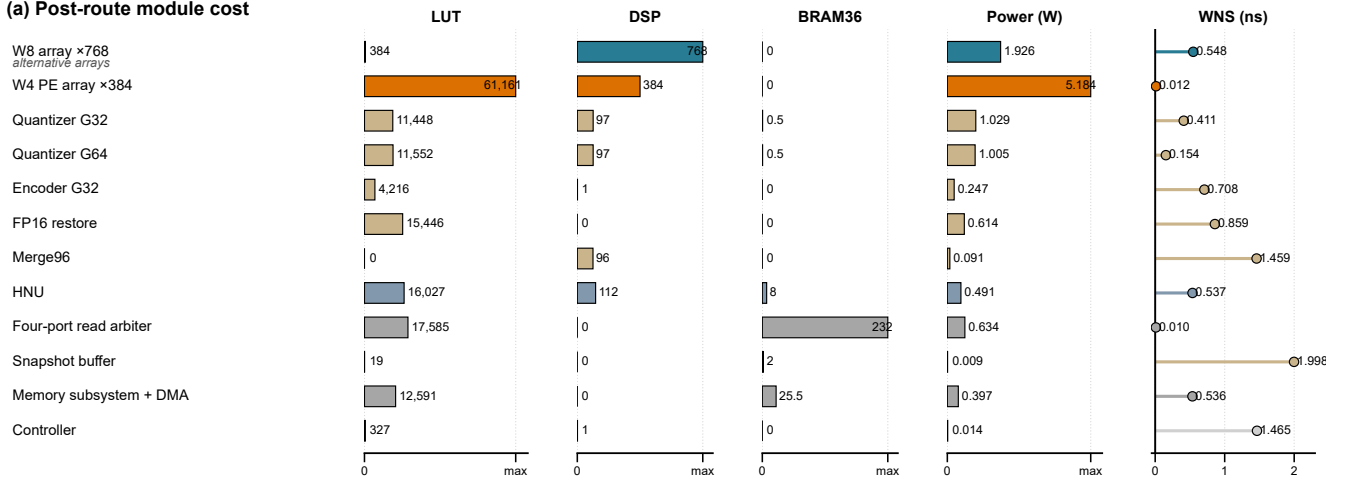
The Pack2 array contains 768 matrix DSPs and generates two products per DSP cycle. Counting a multiplication and accumulation as two operations gives

$$P_{\text{peak}} = 768 \times 2 \times 2 \times 0.300 = 921.6 \text{ GOPS}. \quad (18)$$

TABLE IV
POST-ROUTE MODULE RESOURCES, VECTORLESS DYNAMIC POWER, AND TIMING. BRAM IS IN 36-KBIT EQUIVALENTS. ALL LISTED MODULES MEET THE 3.333-NS OOC
TIMING CONSTRAINT.

Module	LUT	FF	DSP	BRAM	Power (W)	WNS (ns)	Model work category
W8 multiplier cores $\times 768$	384	38,784	768	0	1.926	+0.548	Matrix products
Quantizer G32	11,448	6,817	97	0.5	1.029	0.411	Input quantization
Quantizer G64	11,552	7,358	97	0.5	1.005	0.154	Input quantization
Encoder G32	4,216	3,807	1	0	0.247	0.708	Output encoding
FP16 restore	15,446	7,013	0	0	0.614	0.859	Scale restoration
Merge96	0	2	96	0	0.091	1.459	Group merging
HNU	16,027	20,448	112	8	0.491	0.537	Nonlinear / reduction / vector
Four-port read arbitration	17,585	148	0	232	0.634	+0.010	Local data movement
Snapshot buffer	19	6	0	2	0.009	1.998	Group-result buffering
Memory subsystem (includes DMA)	12,591	4,479	0	25.5	0.397	0.536	Memory transfers
Controller	327	18	1	0	0.014	1.465	Execution control

(a) Post-route module cost



(b) Critical-path attribution for one invocation

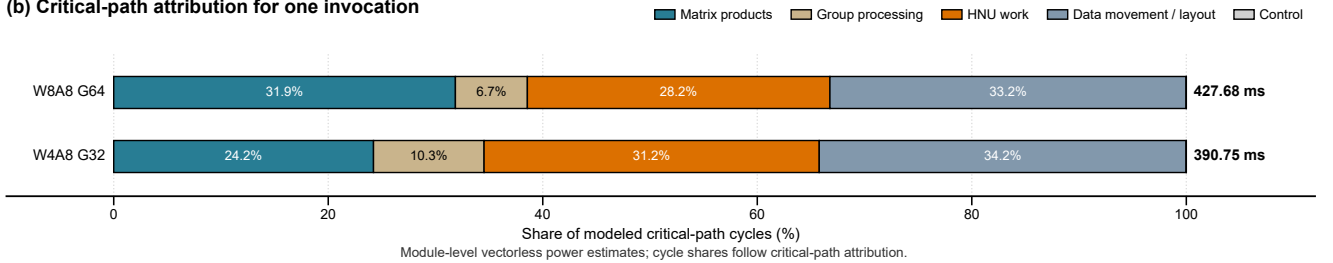


Fig. 12. Module implementation costs and execution-time distribution. (a) Resource and power columns are independently scaled and labeled with their values; WNS is relative to zero slack. (b) Critical-path time is grouped by hardware responsibility.

Combining this peak with the 6.03 W module-based estimate yields 152.84 GOPS/W. The end-to-end evaluation separately reports 427.68 ms per TurboVLA invocation, covering matrix, group-processing, nonlinear, and data-movement work.

Huang *et al.* [13] combine integer-only execution with a group-vector systolic organization for vision transformers. Dong *et al.* [14] exploit mixed-granularity sparsity in window-based transformers, while HeatViT [16] dynamically selects image tokens and reuses backbone hardware for token selection. Vela focuses on a different source of cost: independently scaled reduction groups in a closed-loop VLA policy. Its quantization granularity is selected using robot-task success rates, and its

arithmetic datapath explicitly handles group-result snapshots and scale-aligned merging.

EdgeLLM [15] combines CPU-FPGA execution with mixed-precision operators, group-vector arrays, and overlapped data transfer for large language models. Vela uses W8A8 for both static-weight matrix products and dynamic attention, with an FP16-facing HNU for the heterogeneous operators in TurboVLA. This organization couples packed matrix execution to group quantization, result restoration, and shared nonlinear arithmetic.

The A-SSCC'25 accelerator [17] is the closest application-level comparison and uses the same XCZU7EV device. It

TABLE V
COMPARISON WITH FPGA-BASED TRANSFORMER AND EMBODIED-AI ACCELERATORS.

Metric	TCAS-I'23 [13]	TCAS-I'25 [14]	TCAS-I'25 [15]	HPCA'23 [16]	A-SSCC'25 [17]	This Work
FPGA	XCZU9EG	XCZU50	XCZU37P	XCZU9EG	XCZU7EV	XCZU7EV
Application	General CV	General CV	Language modeling	General CV	Robot control	End-to-end VLA
Model	ViT-S, ViT-B	Swin-Transformer	Qwen-7B	DeiT-S	π_0 action expert	TurboVLA
Precision	INT8	INT8	FP16/INT4	INT8	INT8	W8A8 (G64)
Frequency (MHz)	300	170	140	300	200	300
kLUTs / DSPs	144 / 1,268	472.3 / 2,164	1,303 / 9,024	145 / 1,955	76.5 / 1,599	78.147/1,075 ^a
Power (W)	29.6	18.4	56.8	10.697	8.02	6.03 ^a
Throughput (GOPS)	762.7	830.26	1,260	220.6	903.2	921.6 ^b
Throughput / power (GOPS/W)	25.77	45.12	22.26	20.62	112.69	152.84 ^{a,b}

Prior-work figures follow the comparison reported by Ma *et al.* [17], Fig. 6. ^aThis-work resources and power are module-subtotal estimates: the Table IV W8 configuration with one G64 quantizer and one G32 encoder, plus one 0.60 W static-power contribution. Downstream group accumulators are outside the reported multiplier hierarchy. The 1,075 DSP subtotal includes the 977-DSP ablation budget and 98 quantizer/encoder DSPs. ^bTheoretical matrix peak with two operations per MAC; 152.84 is the ratio of that peak to the 6.03 W module-based estimate.

accelerates the π_0 action expert through behavior prediction and hierarchical tile-aware layer fusion, while vision–language semantic processing supplies its KV inputs. Vela instead spans the end-to-end TurboVLA invocation. The two designs address complementary costs: temporal prediction and fusion reduce action-planning work, whereas grouped quantization, Pack2, and HNU sharing reduce the execution cost of Vela’s multimodal workload.

B. Quantization methods

SmoothQuant transfers activation quantization difficulty to weights through an equivalent channel transformation, enabling W8A8 post-training quantization in language models [5]. AWQ uses calibration activations to identify salient weight channels and selects scaling factors for low-bit, weight-only inference [6]. The present work compares closed-loop W8A8 and W4A8 quality under the same selected robot tasks and implements a W8A8 G64 architecture. Its hardware concern is the runtime production of K -group activation scales and the merge of differently scaled integer partial sums. AWQ’s weight-only W4A16 setting uses a different activation precision from this architecture. The Direct result in Section II isolates grouping granularity within this paper’s numerical reference.

C. VLA inference acceleration

EfficientVLA combines language-layer pruning, task-aware visual-token selection, and feature caching in the diffusion action head [18]. SP-VLA jointly schedules a full VLA and a lightweight action generator, then prunes visual tokens using spatial and semantic importance [19]. Action-aware Dynamic Pruning uses instruction relevance to select tokens and recent action trajectories to control pruning across manipulation stages [20]. These methods reduce the frequency or size of inference work. Vela accelerates the retained operators through grouped quantization, scale-aligned merging, and packed arithmetic.

Asynchronous execution addresses the interaction between inference and robot motion. VLASH predicts the robot state at the future execution time from the preceding action chunk,

aligning new predictions with continued action execution [21]. Its focus is reaction latency and prediction–execution alignment. Vela’s cycle-level evaluation measures the hardware service time of one policy invocation; H1–H3 reduce the cost of the computation that an execution scheduler requests.

D. Embodied action and VLA processors

Algorithm–hardware co-design for robotic computing spans sensing, decision-making, and control [22]. DADU-Corki predicts near-future trajectories to reduce model-invocation frequency, accelerates their conversion to torque commands, and overlaps communication with computation [23]. This changes how often the policy runs and how its output reaches the robot. Vela instead organizes the arithmetic within each invocation, from group quantization to matrix products and nonlinear processing.

Zhang *et al.* present a dedicated diffusion-based robot-manipulation SoC combining speculation and disturbance enhancement, with 94-Hz inference and 7.4-mJ/epoch fine-tuning [24]. Ma *et al.* map behavior prediction and tiled layer fusion to an FPGA action-planning accelerator [17]. Wang *et al.* span the vision–language model and action expert using SimHash-based token filtering and hybrid RRAM–SRAM processing near memory [25]. Together, these designs exploit temporal structure, specialized silicon, and data locality. Vela’s FPGA contribution is the coordinated W8A8 datapath for group scales, packed products, and heterogeneous nonlinear operators.

DiTPA identifies action-, denoising-, and modality-level redundancy in a DiT action planner. It maps action prediction, alternating denoising with feature reuse, and calibrated multimodal skipping to dedicated hardware [26]. Vela focuses on the execution of the complete TurboVLA invocation: H1 coordinates independently scaled products, H2 packs INT8 arithmetic, and H3 serves nonlinear and reduction operators. These mechanisms accelerate retained computation and complement approaches that reduce repeated action-planning work.

E. Diffusion transformer reuse

RADiT checks blockwise similarity across diffusion timesteps and reuses the preceding result when its dynamic

threshold permits [27]. DSTAR combines fine-grained quantization of differential activations in linear layers with sparse attention reuse [28]. Kaleido exploits channel-wise spatial and temporal correlations in video diffusion latents, using partial-result reuse and hardware data dispatch to serve the resulting irregular work [29]. All three reduce repeated DiT computation. Their opportunities depend on timestep or latent correlations; this paper characterizes a specific VLA invocation and its quantization groups without assuming that such reuse is available in the traced workload. In particular, DSTAR’s low-bit differential activation is a different numerical object from the complete group activations quantized by H1.

F. Matrix arrays and heterogeneous operators

DiP changes the input schedule and weight-stationary dataflow of a systolic array to remove synchronization FIFOs [30]. Transitive Array instead constructs a result-reuse graph for binary GEMM and schedules dependent reuse across lanes [31]. DSP-Packing studies how wide FPGA multipliers can perform multiple narrow products and how packing errors are corrected [7]. H2 uses the DSP pre-adder to remove offset terms before multiplication, then sends recovered signed products to group accumulators. H1 supplies scale-aligned merging downstream. The HNU adds shared elementwise PWL arithmetic, row reductions, and an asymmetric set of special-function units for operators outside GEMM. These components target costs that product density, array input scheduling, and result reuse alone do not quantify in this VLA trace.

VII. CONCLUSION

Grouped W8A8 VLA execution couples task quality to scale processing, arithmetic mapping, and heterogeneous operator service. The proposed FPGA architecture addresses these costs through H1 group quantization and merging, H2 Pack2 DSP products, and H3 nonlinear and reduction paths. The W8A8 G64 numerical reference achieves 153/160 successes, matching the floating-point aggregate, while the cycle model gives 427.68 ms per invocation at 300 MHz. The study shows why packed multiplication must be evaluated together with result handling and nonlinear work. Post-route module results quantify the implementation costs of these components.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183. [Online]. Available: <https://proceedings.mlr.press/v229/zitkovich23a.html>
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2679–2713. [Online]. Available: <https://proceedings.mlr.press/v270/kim25c.html>
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, June 2025. [Online]. Available: <https://www.roboticsproceedings.org/rss21/p010.html>
- [4] H. Xie, C. Yao, X. Wu, Y. Zhu, D. Liang, X. Bai, and H. Ding, “TurboVLA: Real-time vision-language-action model at 32 Hz on an RTX 4090 with less than 1 GB VRAM,” *arXiv preprint arXiv:2607.27205v2*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.27205v2>
- [5] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, 2023, pp. 38 087–38 099. [Online]. Available: <https://proceedings.mlr.press/v202/xiao23c.html>
- [6] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of the 7th Conference on Machine Learning and Systems (MLSys)*, 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>
- [7] J. Sommer, M. A. Özkan, O. Kesocze, and J. Teich, “DSP-Packing: Squeezing low-precision arithmetic into FPGA DSP blocks,” *arXiv preprint arXiv:2203.11028*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.11028>
- [8] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo, “CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.19650>
- [9] Z. Hou, T. Zhang, Y. Xiong, H. Duan, H. Pu, R. Tong, C. Zhao, X. Zhu, Y. Qiao, J. Dai, and Y. Chen, “DiTA: Scaling diffusion transformer for generalist vision-language-action policy,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 7686–7697. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2025/html/Hou_Dita_Scaling_Diffusion_Transformer_for_Generalist_Vision-Language-Action_Policy_ICCV_2025_paper.html
- [10] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, J. Gu, Z. Wang, Y. Ding, B. Zhao, D. Wang, and X. Li, “SpatialVLA: Exploring spatial representations for visual-language-action models,” in *Proceedings of Robotics: Science and Systems*, June 2025. [Online]. Available: <https://www.roboticsproceedings.org/rss21/p011.html>
- [11] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.01844>
- [12] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [13] M. Huang, J. Luo, C. Ding, Z. Wei, S. Huang, and H. Yu, “An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 12, pp. 5289–5301, 2023.
- [14] Q. Dong, S. Zhang, and Z. Wang, “An efficient window-based vision transformer accelerator via mixed-granularity sparsity,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 9, pp. 4751–4764, 2025.
- [15] M. Huang, A. Shen, K. Li, H. Peng, B. Li, Y. Su, and H. Yu, “EdgeLLM: A highly efficient CPU-FPGA heterogeneous edge accelerator for large language models,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 7, pp. 3352–3365, 2025.
- [16] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang, and Y. Wang, “HeatViT: Hardware-efficient adaptive token pruning for vision transformers,” in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 442–455.
- [17] Y. Ma, S. He, S. Gong, L. Li, M. Li, X. Xiong, C. Wang, H. Tian, Q. Liu, X. Zeng, C. Chen, and H. Zhu, “A 102.5-Hz real-time FPGA robot-action-planning accelerator with velocity-estimation-driven behavior prediction for embodied artificial intelligence,” in *Proceedings of the IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2025, pp. 76–78.

- [18] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, "EfficientVLA: Training-free acceleration and compression for vision-language-action models," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/3a2ef31a1e45908901adc0ca853a8faf-Abstract-Conference.html
- [19] Y. Li, Y. Meng, Z. Sun, K. Ji, C. Tang, J. Fan, X. Ma, S.-T. Xia, Z. Wang, and W. Zhu, "SP-VLA: A joint model scheduling and token pruning approach for VLA model acceleration," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/hash/4072543747a14bbed76284cf2c04b9e9-Abstract-Conference.html
- [20] X. Pei, Y. Chen, S. Xu, Y. Wang, Y. Shi, and C. Xu, "Action-aware dynamic pruning for efficient vision-language-action manipulation," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/hash/1223ca2b14513106ad55db6a537337f9-Abstract-Conference.html
- [21] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han, "VLASH: Real-time VLAs via future-state-aware asynchronous inference," *arXiv preprint arXiv:2512.01031*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.01031>
- [22] L. Yan, X. Zhao, B. Yang, Y. Wu, G. Dai, J. Li, C.-Y. Tsui, K.-T. Cheng, Y. Zhang, and F. Tu, "Robotic computing system and embodied ai evolution: An algorithm-hardware co-design perspective," *Journal of Semiconductors*, vol. 46, no. 10, p. 101201, 2025. [Online]. Available: <https://www.jos.ac.cn/article/doi/10.1088/1674-4926/25020034>
- [23] Y. Huang, Y. Hao, B. Yu, F. Yan, Y. Yang, F. Min, Y. Han, L. Ma, S. Liu, Q. Liu, and Y. Gan, "DADU-Corki: Algorithm-architecture co-design for embodied ai-powered robotic manipulation," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 327–343. [Online]. Available: <https://doi.org/10.1145/3695053.3731099>
- [24] S. Zhang, X. Chen, C. Peng, H. Yang, Y. Liu, and H. Jia, "A 94Hz inference and 7.4mJ/Epoch fine-tune edge SoC for diffusion-based robot manipulation with speculation and disturbance enhancement," in *2025 Symposium on VLSI Technology and Circuits*, 2025, pp. 1–3. [Online]. Available: <https://doi.org/10.23919/VLSITechnologyandCirc61589.2025.11074779>
- [25] J. Wang, Z. Yang, Y. Chen, Y. Zhang, H. Li, R. Peng, Z. Kong, L. Ran, X. Feng, Z. Lyu, L. Zhao, J. Li, Y. Li, F. Zhou, and L. Lin, "A scalable vision-language-action edge processor with token filtering and hybrid processing-near-memory for 6.7-ms perception-to-action latency in robot control," in *Proceedings of the IEEE Symposium on VLSI Technology and Circuits*, 2026.
- [26] X. Zhao, L. Yan, J. Li, Y. Wu, and F. Tu, "DiTPA: A DiT-based action planner accelerator exploiting action-denoising-multimodality redundancy for embodied artificial intelligence," In *Proceedings of the 53rd ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2026.
- [27] Y. Park, S. Kim, Y. Kim, G. Ji, and S. Ryu, "RADiT: Redundancy-aware diffusion transformer acceleration leveraging timestep similarity," in *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [28] C. Zhang, J. Zhao, Y. Feng, C. Zhang, Q. Chen, and M. Guo, "DSTAR: Accelerating diffusion transformers via spatial and temporal redundancy reduction," *arXiv preprint arXiv:2607.15846*, 2026, pDF identifies MICRO 2026. [Online]. Available: <https://arxiv.org/abs/2607.15846>
- [29] W. Miao, H. Liu, W. Hu, Z. Liu, A. Chen, J. Yu, Y. Yao, Y. Gan, J. Zhao, J. Leng, M. Guo, and Y. Feng, "Kaleido: Algorithm-hardware co-design for video diffusion transformers by exploiting latent space correlations," *arXiv preprint arXiv:2607.13770*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.13770>
- [30] A. J. Abdelmaksoud, S. Agwa, and T. Prodromakis, "DiP: A scalable, energy-efficient systolic array for matrix multiplication acceleration," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 73, no. 1, pp. 283–293, 2026.
- [31] C. Guo, C. Wei, J. Tang, B. Duan, S. Han, H. Li, and Y. Chen, "Transitive array: An efficient GEMM accelerator with result reuse," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025.